The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch25_agents

AYuSh

Contents

Chapter 25: AI Agents & Multi-Agent Systems

An **agent** is what you get when you stop using a language model as a one-shot text transformer and start using it as the *controller* of a loop: it observes a state, decides an action (call a tool, write to memory, answer), executes it, observes the result, and repeats until the task is done. The shift is from *generation* to *control*, and it changes the engineering problem entirely — a single LLM call either works or does not, but an agent is a stochastic process running for many steps, where the interesting quantities are success *as a function of* the step budget, the reliability of each step, the environment's unpredictability, and the cost of the whole trajectory. That is why this chapter is built out of measurements rather than diagrams: the interview questions about agents are quantitative ("why does ReAct need a loop?", "when does planning beat reacting?", "why does reflection sometimes not help?", "when do more agents help?"), and each has a clean answer you can *show*.

Because agents are an application-layer phenomenon — the mechanisms live in the orchestration, not in the weights of any single model — the listings follow the exact-simulation approach of Chapter 23: each is a from-scratch simulation of one agent mechanism over a controlled synthetic environment, with a deliberately simple stochastic "policy" standing in for the LLM so the *system* behavior is isolated and reproducible. This is the honest way to make causal claims about architecture ("*this* is why checkpointing matters") without conflating them with the quality of whatever base model happens to be plugged in. Where a technique's benefit is conditional, the condition is measured, not asserted — reflection is shown helping only when a real error signal is present, and planning is shown winning only when the environment is predictable.

The measurements: a ReAct loop whose success is zero until the step budget reaches the number of hops and then plateaus at the per-step reliability raised to the hop count (0.9^h), the two failure modes of any agent loop (Listing 1); plan-and-execute versus ReAct versus a replanning hybrid across environment surprise, where blind planning is cheapest (1 call) but collapses from 1.00 to 0.03 as surprise rises, ReAct stays robust (0.68) at ~5× the cost, and the hybrid recovers most robustness (0.78) at moderate cost (Listing 2); reflection that climbs 0.30 → 1.00 with a verifier signal but stays flat at 0.31 with blind self-critique (Listing 3); tool selection that collapses from 1.00 to 0.07 as the toolset grows to 128 unless you *retrieve* the relevant tools first (holding 0.52), and function-call validity that only schema-constrained decoding guarantees (Listing 4); a from-scratch state-graph executor where checkpointing bounds recomputation — at a 50% node crash rate, 20 node executions with checkpoints versus 808 without (Listing 5); multi-agent decomposition where a supervisor with fresh workers holds 0.43 success on a 10-subtask job that sinks a single long-context agent to 0.04, plus a Condorcet

voting effect lifting a debate from 0.62 to 0.79 (Listing 6); memory where a fixed context window forgets an early fact the instant it scrolls out (recall → 0) while vector retrieval holds it (0.77 at 200 turns) and a running summary settles at a lossy floor (Listing 7); and agent evaluation where three agents with near-identical final-answer accuracy (0.85/0.80/0.85) are cleanly separated only by trajectory metrics — the lucky guesser's grounded-success is 0.28 and the thorough agent's step-efficiency is 0.40 (Listing 8).

From chatbot to agent: the perceive–decide–act loop

The defining structure of an agent is the loop: given a goal and the current context, the LLM emits an *action*, an external system *executes* it and returns an *observation*, and the observation is appended to the context for the next iteration, until the model emits a terminal "answer" action or a budget is hit. Everything else — tools, memory, planning, multiple agents — is machinery hung on this skeleton. Two properties of the loop dominate its behavior and recur through the chapter. First, **it is sequential and stochastic**, so per-step errors *compound*: a task requiring h correct steps in a row succeeds with probability roughly p^h where p is per-step reliability, which is why long-horizon agents are fragile and why so much agent engineering is about raising p (better tools, verification) or shortening the horizon (decomposition). Second, **it needs a budget and a stopping rule**: too few steps and the agent cannot finish; unbounded steps and a confused agent loops forever burning tokens.

Listing 1 makes both concrete on a multi-hop lookup task whose facts live in an external knowledge base rather than the model's weights (so the agent genuinely *must* use the loop). Success as a function of the step budget shows a sharp cliff — it is exactly zero until the budget reaches the number of hops h, because the agent physically cannot chase h references in fewer than h steps — and then a plateau at p^h (measured 0.90, 0.81, 0.73 for one, two, and three hops at $p = 0.9$), the ceiling set by compounding per-step error. These are the two failure modes every agent question comes back to: *not enough budget* and *not enough per-step reliability*, and no amount of one fixes the other.

Agent architectures: ReAct, plan-and-execute, reflection

Three architectures cover most of what interviews ask. **ReAct** (reason + act, Chapter 23) interleaves a thought, an action, and an observation each iteration, deciding the next action from the *latest* observation — maximally adaptive, because every step reconsiders the situation, but it pays for an LLM call per step and can wander without a global plan. **Plan-and-execute** front-loads the reasoning: the model produces a complete multi-step plan once, then executes the steps (optionally with a cheaper model or no model at all), which slashes cost and latency and gives a legible plan to inspect — but it is brittle, because a plan written before execution assumes the world will behave, and any surprise mid-run derails it with no recovery. The productive middle is **plan-and-execute with**

replanning: run the plan, but when a step's observation contradicts the plan's assumption, pay for one replan and continue.

Listing 2 sweeps the environment's per-step "surprise" rate and measures all three on success and on LLM-call cost. With a predictable environment (surprise ≈ 0) all three succeed and plan-and-execute is cheapest by far (one planning call versus ReAct's one-call-per-step). As surprise rises, blind plan-and-execute collapses (success 1.00 → 0.03) because it cannot react to anything unexpected, while ReAct stays robust (→ 0.68) precisely because it re-decides every step — at roughly five times the call cost. The replanning hybrid captures most of ReAct's robustness (→ 0.78) at a fraction of the cost (rising only to ~3 calls), which is why it is the common production choice. The interview synthesis: *plan when the environment is predictable and cost matters; react when it is not; and replan to get both.*

Reflection is an orthogonal add-on to any of these: after an attempt, the agent critiques its own output and retries, the idea behind Reflexion and self-refine. Its usefulness is entirely conditional on the *quality of the feedback*, which Listing 3 measures by comparing reflection with a real verifier (a unit test, a tool error, a ground-truth check) against blind self-critique with no external signal. With a verifier, cumulative success climbs 0.30 → 1.00 over a few rounds with the expected diminishing returns, because the agent learns which attempts genuinely failed and why. With self-critique *only*, success stays flat at 0.31 — blind reflection rarely fixes a real error and sometimes *overwrites a correct answer*, a net wash. This is the single most important thing to know about reflection: it is a way to *exploit* an error signal, not a way to *create* one, so an agent that cannot tell whether it succeeded gains nothing from reflecting.

Tool use and function calling

Tools are how an agent acts on the world and grounds itself in facts outside its weights — search, code execution, database queries, API calls. The mechanism is **function calling**: the model is shown a set of tool schemas (name, description, typed arguments) and, instead of prose, emits a structured call naming a tool and filling its arguments, which an external runtime executes before returning the result. Two distinct reliability problems live here, and Listing 4 separates them. The first is **tool selection**: with a handful of tools the model reliably picks the right one, but as the toolset grows the choice becomes a needle-in-a-haystack over a long, confusable in-context list, and selection accuracy collapses (measured 1.00 at 2 tools down to 0.07 at 128). The fix is **tool retrieval** — embed the query, retrieve the top-m relevant tool schemas, and let the model choose only among those — which keeps accuracy far higher (0.52 at 128 tools) by bounding the choice, and is the standard pattern once an agent has more than a few dozen tools. The second problem is **argument validity**: even with the right tool chosen, the emitted call must satisfy the schema, and unconstrained generation drops a valid-JSON call as per-token error rises (a single bad token breaks the parse), whereas schema-**constrained decoding** (Chapter 23) is valid by construction. Because an end-to-end successful call

requires *both* the right tool and valid arguments, the two probabilities multiply — the listing's naive-plus-unconstrained pipeline lands at 0.04 where retrieve-plus-constrained reaches 0.57.

The emerging standard for *how* tools are exposed to models is the **Model Context Protocol (MCP)**, an open protocol that standardizes the interface between an LLM application (the *host/client*) and external capability *servers* that expose **tools** (callable functions), **resources** (readable data), and **prompts** (reusable templates) over a uniform JSON-RPC transport. Its point is decoupling: before MCP, every agent framework integrated every tool with bespoke glue; with MCP, any MCP-compatible client can use any MCP server, so a filesystem server, a GitHub server, or a database server is written once and reused everywhere. Conceptually it is USB-C for tool use — a common connector — and it is worth being able to name its three primitives (tools, resources, prompts) and the client–server split in an interview, because it is quickly becoming the default way agents acquire capabilities.

Orchestration: chains, graphs, state, and checkpointing

The frameworks that structure agents — LangChain and, for stateful control flow, LangGraph — exist because raw agent loops become unmanageable as they grow branches, retries, and multiple actors. The foundational abstraction is the **chain**: a directed sequence of steps (prompt → LLM → parse → tool → prompt …) composed so the output of each feeds the next, which is enough for linear pipelines but cannot express loops or conditional branching cleanly. **LangGraph** generalizes the chain to a **graph**: nodes are functions that read and write a shared, typed **state** object, and edges — including **conditional edges** that route based on state — connect them, so cycles (retry until a check passes), branches (route by intent), and fan-out/fan-in (parallel workers) are all first-class. Modeling an agent as an explicit state machine is what makes complex control flow legible and testable, and it is the reason graph frameworks displaced ad-hoc loops for anything nontrivial.

The state abstraction buys one more thing that matters enormously in production: **checkpointing**, persisting the state after each node so a run can be *resumed* rather than restarted after an interruption — a crashed tool, a rate limit, a human-in-the-loop pause, or a process restart. Listing 5 builds a small graph executor from scratch (four nodes, a conditional cycle from a verify step back to retrieval, a shared state dict) and measures why checkpointing is not optional under real failure rates. Without checkpointing, a crash restarts the whole graph and recomputes every prior node, so total work explodes super-linearly as the crash rate rises — from 7 node executions with no crashes to **808** at a 50% per-node crash rate. With checkpointing, a crash resumes from the last completed node, so wasted work stays bounded — **20** executions at the same 50% crash rate. The same persisted state also underpins human-in-the-loop approval (pause at a node, wait for a human, resume) and time-travel debugging (replay from any checkpoint), which is why durable state is the backbone of production agent frameworks rather than a nice-to-have.

Multi-agent systems

Once one agent works, the question is whether *several* specialized agents do better than one generalist, and the honest answer is: sometimes, for two specific and separable reasons that Listing 6 isolates. The first is **decomposition under context limits**. A single agent handling a task with many sub-tasks in one growing context suffers interference — the context fills with intermediate work, attention spreads thin, and per-sub-task reliability degrades as the job lengthens; the listing's single agent sinks from 0.92 on a one-sub-task job to 0.04 on a ten-sub-task job. A **supervisor–worker** pattern — an orchestrator that routes each sub-task to a *fresh* worker with a clean, focused context and aggregates the results — keeps per-sub-task reliability constant, so end-to-end success decays far more slowly (0.92 → 0.43 over the same range). This is the core argument for multi-agent structure: it is a way to give each piece of work an uncluttered context and a specialized instruction set, at the cost of orchestration overhead and more model calls.

The second reason is **aggregation**: independent agents answering the same question and voting is a Condorcet-jury effect (Chapter 23's self-consistency, lifted to the agent level). The listing's majority vote over k agents, each correct 62% of the time, rises to 0.79 at eleven agents — provided the precondition holds that each agent is better than chance and their errors are *independent* (correlated agents, e.g. the same model with the same prompt, do not decorrelate and gain little). Beyond these, the standard patterns worth naming are **debate** (agents critique each other's answers over rounds), **hierarchical teams** (supervisors of supervisors), and role-specialization (planner, coder, critic, researcher). The interview caveat is symmetry to the benefits: every extra agent multiplies cost and latency, adds communication surface where errors and hallucinations *propagate* between agents, and needs a coordination protocol — so multi-agent is justified when a task genuinely decomposes or benefits from diverse votes, not as a default.

Memory: short-term, long-term, and episodic

An agent's raw memory is its **context window** — the tokens currently in front of the model — and that is **short-term memory**: fast, fully attended, but bounded and *volatile*, because anything beyond the window is simply gone. Listing 7 shows the consequence starkly: probing recall of a fact introduced early in a conversation, a fixed window holds it perfectly until the conversation length exceeds the window and then recall falls off a cliff to zero — the fact scrolled out. **Long-term memory** solves this by writing everything to an external store (typically a vector database) and *retrieving* the relevant pieces on demand — RAG (Chapter 24) applied to the agent's own history — so an early fact stays recoverable regardless of conversation length; the listing's retrieval memory holds recall at 0.77 even at 200 turns, decaying only slowly as accumulated history adds distractors. **Episodic memory** — a running, compressed *summary* of past turns or past task episodes — is the cheaper middle ground: it does not grow with history and needs no retrieval call,

but it is *lossy*, so recall of any specific early fact settles at a floor (measured ~0.42) rather than staying perfect.

The taxonomy interviews expect maps onto these: **short-term / working memory** (the context window and a scratchpad within a single task), **long-term memory** split into *semantic* (facts and knowledge, often the retrieval store), *episodic* (records of past interactions and their outcomes, replayed to inform new ones — the basis of experience-based improvement), and *procedural* (learned skills/routines, often baked into the system prompt or tools). Production agents use them in combination: the window for the active task, a summary to compress the recent past cheaply, and a vector store for durable recall — with a memory-management policy deciding what to write, what to summarize, and what to retrieve, which is itself an active area and a rich source of interview discussion (what to store, how to avoid retrieval distraction, how to forget).

Evaluating and observing agents

Agents are harder to evaluate than single LLM calls because success is a property of a *trajectory*, not a single output, and the same final answer can be reached by a sound process or a lucky accident. Relying on **final-answer accuracy** alone is therefore misleading, which Listing 8 demonstrates by scoring three agent profiles that share nearly identical final-answer accuracy (0.85, 0.80, 0.85) on four metrics. The final-answer metric ranks all three as roughly equivalent, but the **trajectory metrics** separate them decisively: the "lucky guesser" — right answers reached by calling the wrong tools and skipping steps — has a tool-call precision of 0.40 and a *grounded-success* rate (correct **and** via a valid trajectory) of just 0.28, meaning most of its wins are unreproducible and unsafe; the "thorough but slow" agent is well-grounded but has a step-efficiency of 0.40, meaning it burns two-and-a-half times the necessary calls. Neither pathology is visible in the final-answer number.

This is why **observability** — capturing the full trace of every thought, tool call, argument, observation, and token cost — is the foundation of agent engineering, and why tracing tools (LangSmith, LangFuse, and OpenTelemetry-based tracing) are standard infrastructure rather than a debugging luxury. The metrics that matter are componentized along the trajectory: **task success** (end-to-end), **step/trajectory efficiency** (steps taken versus optimal, and token/dollar cost), **tool-call accuracy** (right tool, valid arguments, correct results), **grounding/faithfulness** of intermediate reasoning, and, for the retrieval an agent does, the RAGAS-style metrics of Chapter 24. Evaluation methods combine curated task suites with programmatic checks where possible (a unit test, an exact-match, a tool-error signal — the same reliable signals that make reflection work) and LLM-as-judge for open-ended steps (with its biases from Chapter 23). The meta-lesson interviews reward: measure the *trajectory*, instrument everything, and never trust a single end-to-end number to tell you whether an agent is actually working.

Code implementations

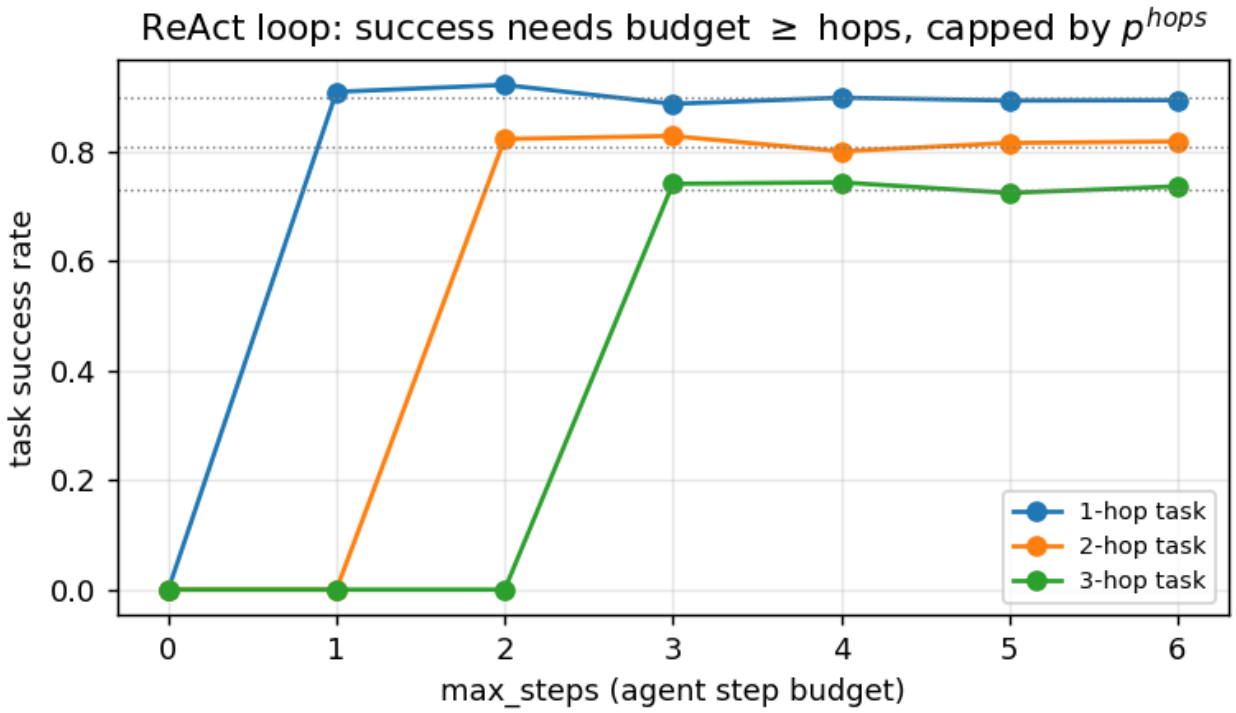
(Each listing is a from-scratch simulation of one agent mechanism over a controlled environment, with a simple stochastic policy standing in for the LLM so the system *behavior — not a particular model's quality — is what is measured. The numbers in the prose are these programs' actual output; every listing is self-contained.)*

Listing 1 — The ReAct loop: budget and per-step reliability

A multi-hop lookup where facts live outside the weights. Success is zero until the step budget reaches the hop count, then plateaus at the per-step reliability raised to the hops — the two failure modes of any agent loop.

```
"""Listing 1: the ReAct loop and why it needs enough steps AND reliable steps.
Environment: a chain
of entities e0 -> e1 -> ... where each fact ('what does e_i link to?') lives in an
external knowledge
base, NOT in the agent's weights. A query asks the agent to follow the link h times and
report the
final attribute. The agent runs a ReAct loop: think -> look up current entity ->
advance, up to
max_steps, then answer. Each lookup+reasoning step succeeds with probability p
(imperfect tool use);
an h-hop task needs h correct steps in a row, so success is bounded by BOTH the step
budget
(max_steps >= h) and compounding per-step error (~ p^h). We Monte-Carlo the success
surface."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
FIG="figures/ch25"; import os; os.makedirs(FIG,exist_ok=True)
rng=np.random.default_rng(0)
def make_chain(h, nfill=200):
    # entities 0..N; each has a 'link' to another entity and an attribute. Build a
clean h-chain.
    chain=list(range(h+1)); link={}; attr={}
    for i in range(h): link[chain[i]]=chain[i+1]
    for e in range(h+1+nfill): attr[e]=rng.integers(1000)
    # add distractor links for filler entities (dead ends)
    for e in range(h+1, h+1+nfill): link[e]=rng.integers(h+1+nfill)
    return chain, link, attr
def react_episode(h, max_steps, p):
    chain, link, attr = make_chain(h)
    cur = chain[0]; steps=0
    while steps < max_steps:
        if cur == chain[h]:                          # reached the target: answer
            return attr[cur]==attr[chain[h]]
        # one think+lookup+advance step; succeeds w.p. p, else takes a wrong link
(derailed)
        if rng.random() < p: cur = link[cur]
        else: cur = rng.integers(h+1, h+1+50)      # wrong branch -> lost
        steps += 1
    return cur == chain[h]                           # ran out of budget
def success(h, max_steps, p, n=1500):
    return np.mean([react_episode(h,max_steps,p) for _ in range(n)])
P=0.9; steps=list(range(0,7)); hops=[1,2,3]
res={h:[success(h,s,P) for s in steps] for h in hops}
for h in hops:
    print(f"hop={h}: success vs max_steps "+" ".join(f"{s}:{res[h][s]:.2f}" for s in
steps)+f"   (p^{h}={P**h:.3f})")
plt.figure(figsize=(6,3.6))
for h in hops:
    plt.plot(steps,res[h],"o-",label=f"{h}-hop task")
```

```
    plt.axhline(P**h,ls=":",color="gray",lw=0.8)
plt.xlabel("max_steps (agent step budget)"); plt.ylabel("task success rate")
plt.title("ReAct loop: success needs budget $\\geq$ hops, capped by $p^{hops}$")
plt.legend(fontsize=8); plt.grid(alpha=.3); plt.tight_layout()
plt.savefig(f"{FIG}/l1_react.png",dpi=130); print("saved l1_react.png")
```

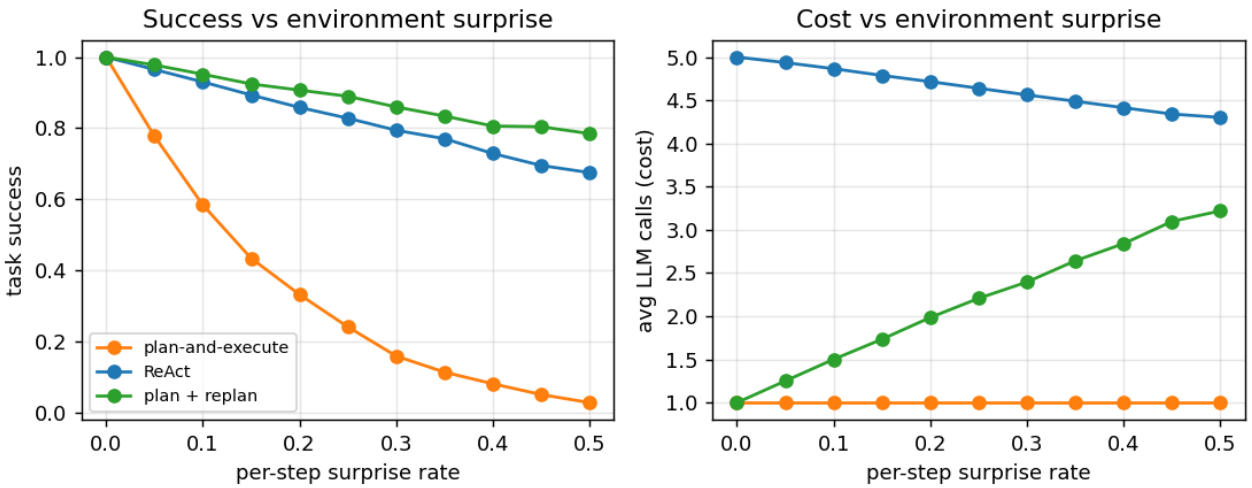


Listing 2 — Plan-and-execute vs ReAct vs replanning

Sweeping environment surprise: blind planning is cheapest but brittle, ReAct is robust but costly, and the replanning hybrid recovers most robustness at moderate cost.

```
"""Listing 2: plan-and-execute vs ReAct vs a replanning hybrid. A task is a fixed h-
step procedure.
PLAN-AND-EXECUTE makes the whole plan up front (1 planning call) then runs it blindly
(h cheap
executes) -- cheap and low-latency, but any mid-run SURPRISE (a step whose result
differs from what
the plan assumed) derails it with no recovery. REACT decides each step from the latest
observation
(h reasoning calls) -- costlier, but adapts to surprises. The hybrid PLAN-AND-EXECUTE +
REPLAN runs
the plan but, on a detected surprise, pays for one replan and continues. We sweep the
per-step
surprise rate and report success and LLM-call cost."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt, os
FIG="figures/ch25"; os.makedirs(FIG,exist_ok=True); rng=np.random.default_rng(0)
H=5   # steps in the procedure
def episode(mode, s):
    calls=0; step=0
    if mode in ("plan","replan"): calls+=1           # upfront plan
    while step<H:
        surprise = rng.random()<s
        if mode=="react":
            calls+=1                                 # a reasoning call each step ->
```

```
always adapts
            if surprise and rng.random()<0.15: return False,calls   # occasionally mis-
adapts
        elif mode=="plan":
            if surprise: return False,calls         # blind plan derails on any
surprise
        elif mode=="replan":
            if surprise:
                calls+=1                            # pay one replan, then continue
                if rng.random()<0.10: return False,calls
        step+=1
    return True,calls
def run(mode,s,n=3000):
    r=[episode(mode,s) for _ in range(n)]
    return np.mean([a for a,_ in r]), np.mean([c for _,c in r])
ss=np.linspace(0,0.5,11); modes=["plan","react","replan"]; lab={"plan":"plan-and-
execute","react":"ReAct","replan":"plan + replan"}
succ={m:[] for m in modes}; cost={m:[] for m in modes}
for m in modes:
    for s in ss:
        a,c=run(m,s); succ[m].append(a); cost[m].append(c)
for m in modes:
    print(f"{lab[m]:20s} surprise=0.0 succ={succ[m][0]:.2f} cost={cost[m][0]:.1f} |
surprise=0.5 succ={succ[m][-1]:.2f} cost={cost[m][-1]:.1f}")
fig,ax=plt.subplots(1,2,figsize=(8.6,3.5))
col={"plan":"#ff7f0e","react":"#1f77b4","replan":"#2ca02c"}
for m in modes: ax[0].plot(ss,succ[m],"o-",color=col[m],label=lab[m])
ax[0].set_xlabel("per-step surprise rate"); ax[0].set_ylabel("task success");
ax[0].set_title("Success vs environment surprise")
ax[0].legend(fontsize=8); ax[0].grid(alpha=.3)
for m in modes: ax[1].plot(ss,cost[m],"o-",color=col[m],label=lab[m])
ax[1].set_xlabel("per-step surprise rate"); ax[1].set_ylabel("avg LLM calls (cost)");
ax[1].set_title("Cost vs environment surprise"); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.savefig(f"{FIG}/l2_plan.png",dpi=130); print("saved
l2_plan.png")
```

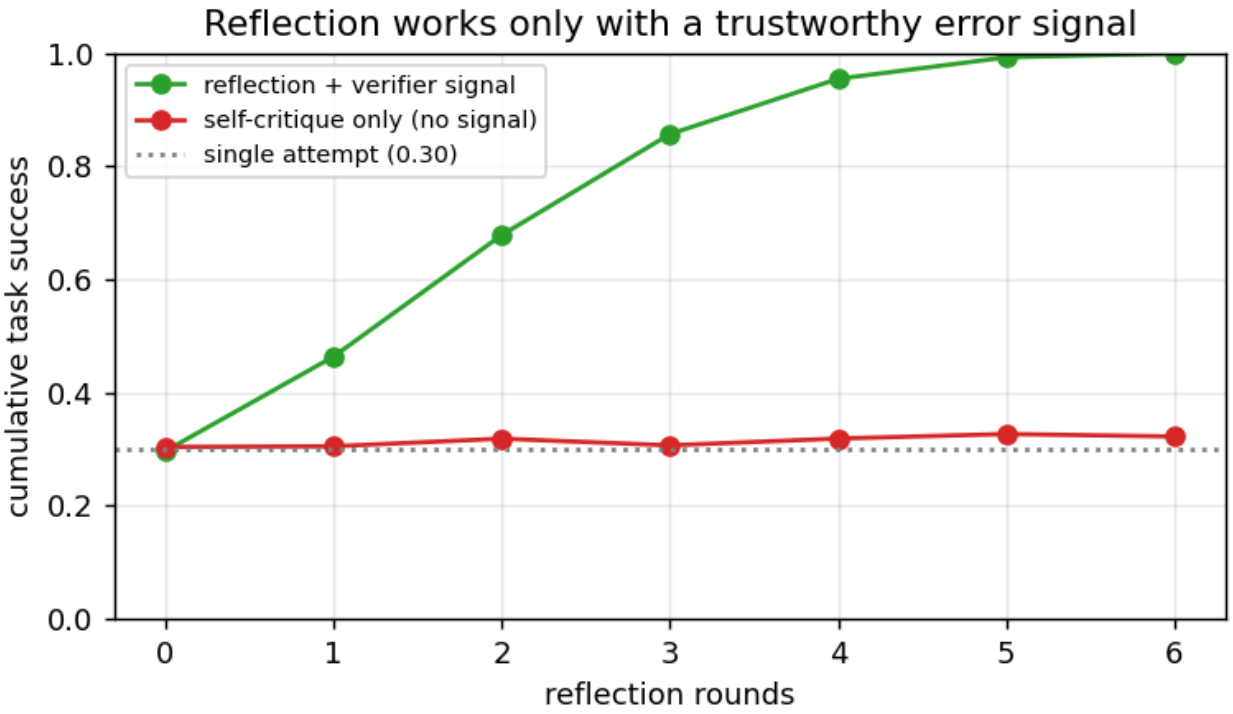


Listing 3 — Reflection needs a real error signal

Reflexion-style retry with self-critique. With a verifier signal, success climbs to 1.00; with blind self-critique it stays flat and sometimes overwrites a correct answer.

```
"""Listing 3: reflection (Reflexion) helps only with a trustworthy error signal. An
agent attempts a
task (base success p0), then iteratively self-critiques and retries. With a VERIFIER
```

```
(unit test, tool
error, ground-truth check) it knows WHICH attempts failed and reflects on the real
error, so each
retry improves and cumulative success climbs with diminishing returns. With SELF-
CRITIQUE ONLY (no
external signal) the agent cannot tell success from failure: blind reflection rarely
fixes a genuine
error and sometimes OVERWRITES a correct answer, so success plateaus or drifts down. We
Monte-Carlo
both."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt, os
FIG="figures/ch25"; os.makedirs(FIG,exist_ok=True); rng=np.random.default_rng(0)
p0=0.30
def episode(mode, rounds):
    correct = rng.random() < p0
    for r in range(rounds):
        if mode=="verifier":
            if correct: break                       # verifier passes -> stop (keep the
win)
            if rng.random() < min(0.85, 0.25+0.15*r): correct=True   # reflect on the
real error
        else:  # self-critique only, no ground-truth
            if not correct:
                if rng.random() < 0.06: correct=True
# occasionally stumbles into a fix
            else:
                if rng.random() < 0.12: correct=False           # "fixes" a correct
answer -> regression
    return correct
def curve(mode, R=6, n=6000):
    return [np.mean([episode(mode,r) for _ in range(n)]) for r in range(R+1)]
ver=curve("verifier"); self_=curve("self")
print("rounds     :", list(range(len(ver))))
print("verifier   :", [f"{x:.2f}" for x in ver])
print("self-only  :", [f"{x:.2f}" for x in self_])
plt.figure(figsize=(6,3.6))
plt.plot(range(len(ver)),ver,"o-",color="#2ca02c",label="reflection + verifier signal")
plt.plot(range(len(self_)),self_,"o-",color="#d62728",label="self-critique only (no
signal)")
plt.axhline(p0,ls=":",color="gray",label=f"single attempt ({p0:.2f})")
plt.xlabel("reflection rounds"); plt.ylabel("cumulative task success"); plt.ylim(0,1)
plt.title("Reflection works only with a trustworthy error signal")
plt.legend(fontsize=8); plt.grid(alpha=.3); plt.tight_layout()
plt.savefig(f"{FIG}/l3_reflect.png",dpi=130); print("saved l3_reflect.png")
```

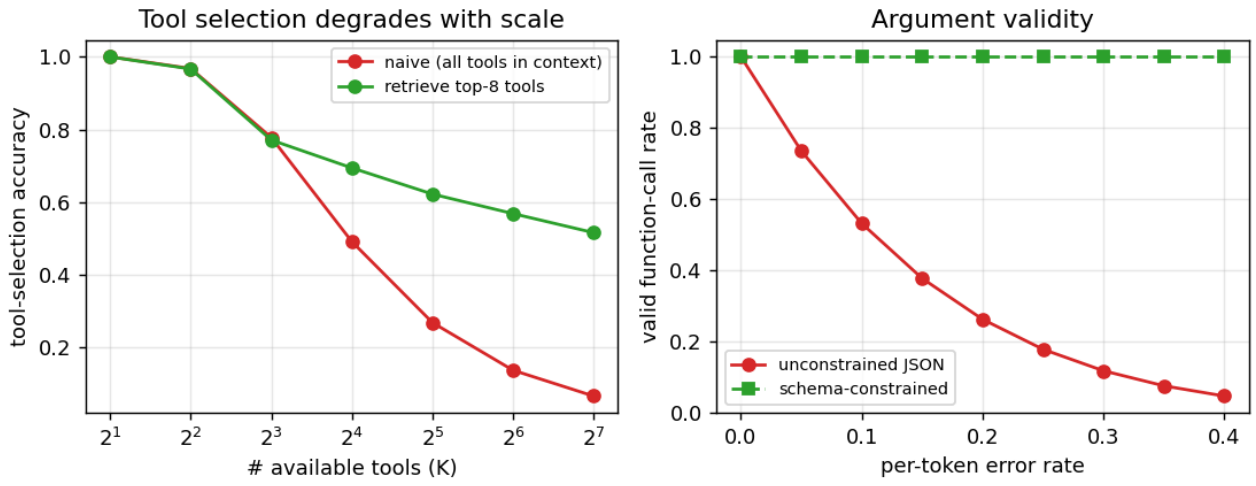


Listing 4 — Tool selection at scale and function-call validity

Tool selection collapses as the toolset grows unless you retrieve the relevant tools first; and only schema-constrained decoding guarantees valid arguments. End-to-end success is the product of both.

```
"""Listing 4: tool use and function calling have two distinct reliability problems. (A) TOOL
SELECTION: as the number of available tools grows, picking the right one from a long in-context list
degrades, because descriptions get confusable and the choice is a needle-in-haystack -- the fix is
tool RETRIEVAL (embed the query, shortlist the top-m tools, choose among those). (B) ARGUMENT
VALIDITY: even with the right tool, the call must be schema-valid; unconstrained generation drops a
valid-JSON call as per-token error rises, while schema-constrained decoding is valid by construction.
End-to-end call success = P(right tool) x P(valid args), so both matter and they compound."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt, os
FIG="figures/ch25"; os.makedirs(FIG,exist_ok=True); rng=np.random.default_rng(0)
d=32
def tool_bank(K):
    T=rng.standard_normal((K,d)); return T/np.linalg.norm(T,axis=1,keepdims=True)
def select_acc(K, retrieve_m=None, n=3000, noise=0.13):
    hit=0
    for _ in range(n):
        T=tool_bank(K); g=rng.integers(K)
        q=T[g]+noise*rng.standard_normal(d); q/=np.linalg.norm(q)
        sims=T@q
        conf=0.11                                              # per-item in-context confusion scale
        if retrieve_m and retrieve_m<K:
            cand=np.argsort(-sims)[:retrieve_m]                # clean-embedding
```

```
shortlist (fixed size m)
            pick=cand[np.argmax(sims[cand] +
rng.standard_normal(len(cand))*conf*np.log2(retrieve_m))]
        else:
            pick=np.argmax(sims + rng.standard_normal(K)*conf*np.log2(max(K,2)))  #
confusion grows with K
        hit += (pick==g)
    return hit/n
Ks=[2,4,8,16,32,64,128]
naive=[select_acc(K) for K in Ks]
retr =[select_acc(K, retrieve_m=8) for K in Ks]
for K,a,b in zip(Ks,naive,retr): print(f"K={K:4d}: naive select {a:.
2f} | retrieve-top8 {b:.2f}")
# (B) argument validity vs per-token error, unconstrained vs schema-constrained
errs=np.linspace(0,0.4,9); Largs=6
unc=[(1-e)**Largs for e in errs]; con=[1.0 for _ in errs]
sel64=select_acc(64); sel64r=select_acc(64,retrieve_m=8)
print(f"\nend-to-end call success @K=64, err=0.2:
naive+unconstrained={sel64*(0.8**Largs):.2f}  "
      f"retrieve+constrained={sel64r*1.0:.2f}")
fig,ax=plt.subplots(1,2,figsize=(8.6,3.5))
ax[0].plot(Ks,naive,"o-",color="#d62728",label="naive (all tools in context)")
ax[0].plot(Ks,retr,"o-",color="#2ca02c",label="retrieve top-8 tools")
ax[0].set_xscale("log",base=2); ax[0].set_xlabel("# available tools (K)");
ax[0].set_ylabel("tool-selection accuracy")
ax[0].set_title("Tool selection degrades with scale"); ax[0].legend(fontsize=8);
ax[0].grid(alpha=.3)
ax[1].plot(errs,unc,"o-",color="#d62728",label="unconstrained JSON")
ax[1].plot(errs,con,"s--",color="#2ca02c",label="schema-constrained")
ax[1].set_xlabel("per-token error rate"); ax[1].set_ylabel("valid function-call rate")
ax[1].set_title("Argument validity"); ax[1].legend(fontsize=8); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.savefig(f"{FIG}/l4_tools.png",dpi=130); print("saved
l4_tools.png")
```

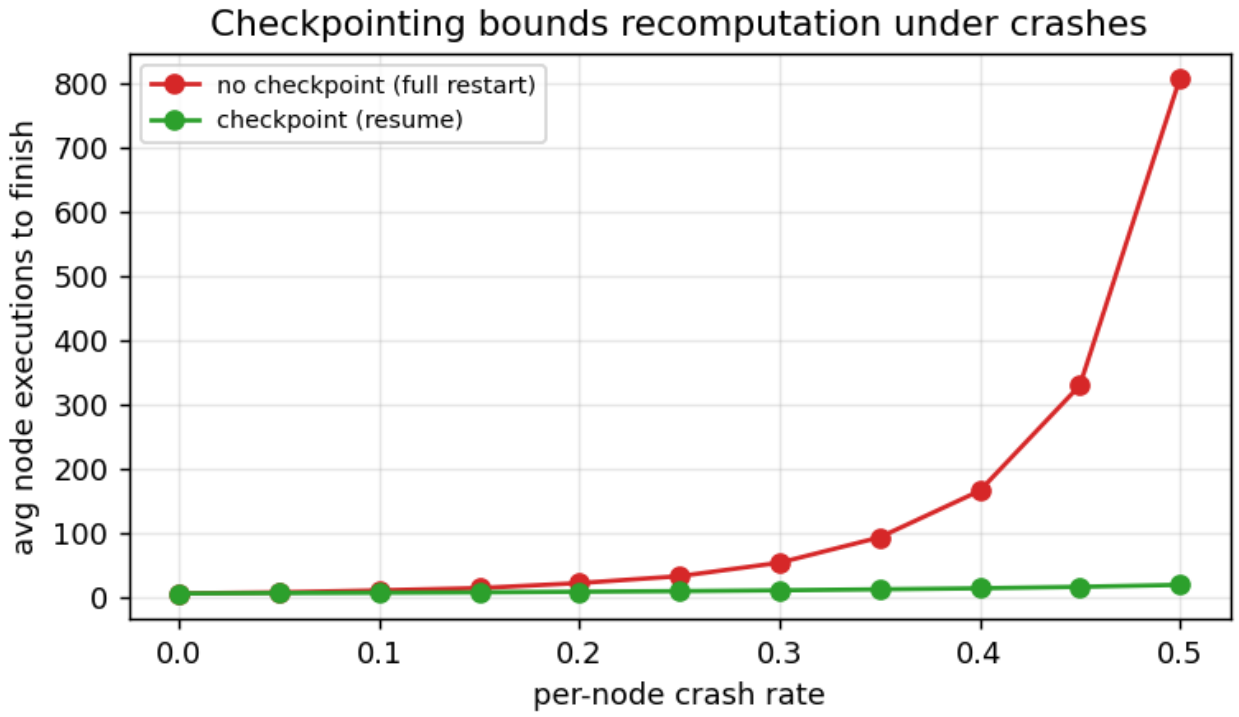


Listing 5 — A state-graph executor with checkpointing

A from-scratch LangGraph-style executor with nodes, a shared state, a conditional cycle, and checkpointing — which bounds recomputation from 808 node executions to 20 at a 50% crash rate.

```
"""Listing 5: an agent as a state-machine graph (the LangGraph model) with
checkpointing. We build a
tiny graph executor from scratch: NODES mutate a shared STATE dict, EDGES route
control, and a
```

```
CONDITIONAL edge creates a cycle (verify -> retrieve on failure, up to a cap, else
end). Real agents
crash mid-run (a tool errors, the process dies). CHECKPOINTING persists state after
each node so a
restart RESUMES from the last completed node; without it, a crash restarts the whole
graph and
recomputes everything. We measure total node executions (wasted work) as the per-node
crash rate
rises."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt, os
FIG="figures/ch25"; os.makedirs(FIG,exist_ok=True); rng=np.random.default_rng(0)
# --- graph definition: node -> function(state)->state; router decides next node ---
def n_plan(s): s["plan"]=True; return s
def n_retrieve(s): s["docs"]=s.get("docs",0)+1; return s
def n_generate(s): s["draft"]=s["docs"]; return s
def n_verify(s): s["ok"]= s["docs"]>=s["need"]; return s     # needs enough retrieval
passes to pass
NODES={"plan":n_plan,"retrieve":n_retrieve,"generate":n_generate,"verify":n_verify}
def router(node,s):
    return {"plan":"retrieve","retrieve":"generate","generate":"verify"}.get(node) or \
           ("end" if s["ok"] or s["loops"]>=s["cap"] else "retrieve_loop")
def run_graph(crash, checkpoint):
    s={"need":int(rng.integers(1,4)),"cap":5,"loops":0,"docs":0,"ok":False}
    order=["plan","retrieve","generate","verify"]; execs=0
    ckpt=None; idx=0
    while True:
        node=order[idx]
        # execute node; may crash
        execs+=1
        if rng.random()<crash:
            if checkpoint and ckpt is not None:
                s=dict(ckpt["state"]); idx=ckpt["idx"]            # resume from last
checkpoint
            else:
                s={"need":s["need"],"cap":5,"loops":0,"docs":0,"ok":False}; idx=0  #
full restart
            continue
        s=NODES[node](s)
        if checkpoint: ckpt={"state":dict(s),"idx":idx}
        nxt=router(node,s)
        if nxt=="end": return execs
        if nxt=="retrieve_loop": s["loops"]+=1; idx=order.index("retrieve"); continue
        idx=order.index(nxt)
def avg_execs(crash,checkpoint,n=4000): return np.mean([run_graph(crash,checkpoint)
for _ in range(n)])
crashes=np.linspace(0,0.5,11)
nock=[avg_execs(c,False) for c in crashes]; ck=[avg_execs(c,True) for c in crashes]
for c,a,b in zip(crashes,nock,ck):
    if abs(c*10-round(c*10))<1e-9 and int(round(c*10))%1==0 and c in
(crashes[0],crashes[4],crashes[-1]):
        print(f"crash={c:.2f}: no-checkpoint execs={a:.1f}  checkpoint execs={b:.1f}")
plt.figure(figsize=(6,3.6))
plt.plot(crashes,nock,"o-",color="#d62728",label="no checkpoint (full restart)")
plt.plot(crashes,ck,"o-",color="#2ca02c",label="checkpoint (resume)")
plt.xlabel("per-node crash rate"); plt.ylabel("avg node executions to finish")
plt.title("Checkpointing bounds recomputation under crashes")
plt.legend(fontsize=8); plt.grid(alpha=.3); plt.tight_layout()
plt.savefig(f"{FIG}/l5_graph.png",dpi=130); print("saved l5_graph.png")
```

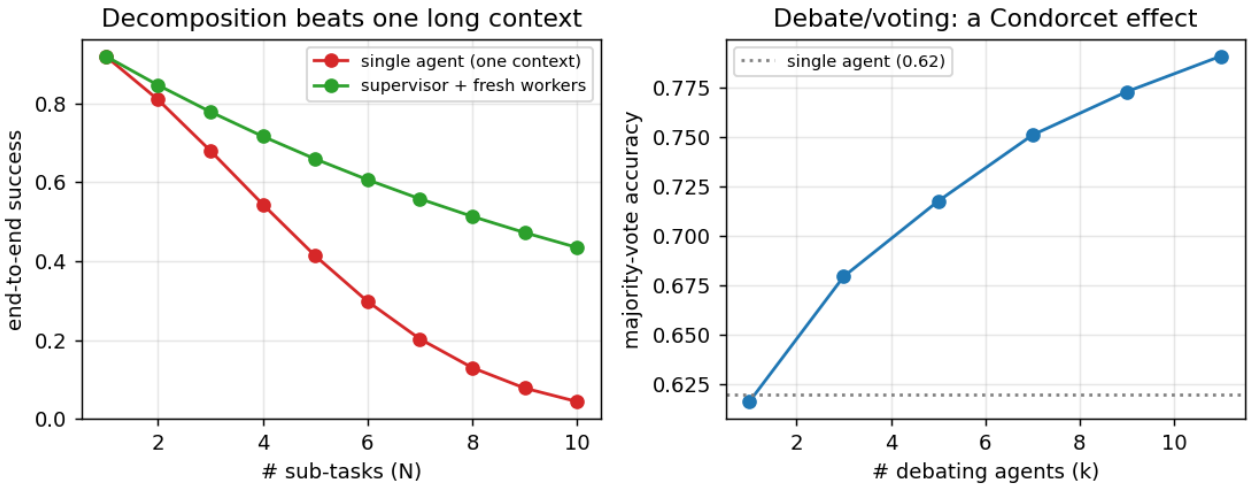


Listing 6 — Multi-agent: decomposition and voting

A supervisor routing sub-tasks to fresh workers holds success where a single long-context agent collapses; and independent agents voting is a Condorcet effect.

```
"""Listing 6: when do multiple agents beat one? Two orthogonal mechanisms. (A)
DECOMPOSITION: a task
with N sub-tasks handled by ONE agent in a single growing context suffers interference
-- per-subtask
success falls as N rises. A SUPERVISOR that routes each sub-task to a FRESH worker with
a clean,
focused context keeps per-subtask success constant, so end-to-end success (needs all N)
decays far
more slowly -- at the cost of N worker calls plus supervisor overhead. (B) DEBATE/
VOTING: k agents
each answer a noisy question and majority-vote -- a Condorcet effect that raises
accuracy when each
agent is better than chance and errors are independent."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt, os
FIG="figures/ch25"; os.makedirs(FIG,exist_ok=True); rng=np.random.default_rng(0)
p0=0.92; beta=0.04
Ns=list(range(1,11))
single=[np.prod([max(0.05,p0-beta*i) for i in range(N)]) for N in Ns]   # context
interference grows
superv=[p0**N for N in Ns]                                              # fresh worker
per subtask
for N in [1,5,10]:
    print(f"N={N:2d} subtasks: single-agent {single[N-1]:.2f}  supervisor-worker
{superv[N-1]:.2f}  (workers={N})")
# (B) debate / majority vote, each agent correct w.p. pa, independent errors
def vote_acc(k,pa,n=20000):
    votes=(rng.random((n,k))<pa).sum(1)
    return np.mean(votes> k/2) + 0.5*np.mean(votes==k/2)   # ties count as coin flip
ks=[1,3,5,7,9,11]; pa=0.62; vacc=[vote_acc(k,pa) for k in ks]
print("debate (pa=0.62): k="+" ".join(f"{k}:{a:.2f}" for k,a in zip(ks,vacc)))
```

```
fig,ax=plt.subplots(1,2,figsize=(8.6,3.5))
ax[0].plot(Ns,single,"o-",color="#d62728",label="single agent (one context)")
ax[0].plot(Ns,superv,"o-",color="#2ca02c",label="supervisor + fresh workers")
ax[0].set_xlabel("# sub-tasks (N)"); ax[0].set_ylabel("end-to-end success");
ax[0].set_title("Decomposition beats one long context")
ax[0].legend(fontsize=8); ax[0].grid(alpha=.3)
ax[1].plot(ks,vacc,"o-",color="#1f77b4");
ax[1].axhline(pa,ls=":",color="gray",label=f"single agent ({pa})")
ax[1].set_xlabel("# debating agents (k)"); ax[1].set_ylabel("majority-vote accuracy");
ax[1].set_title("Debate/voting: a Condorcet effect")
ax[1].legend(fontsize=8); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.savefig(f"{FIG}/l6_multi.png",dpi=130); print("saved
l6_multi.png")
```

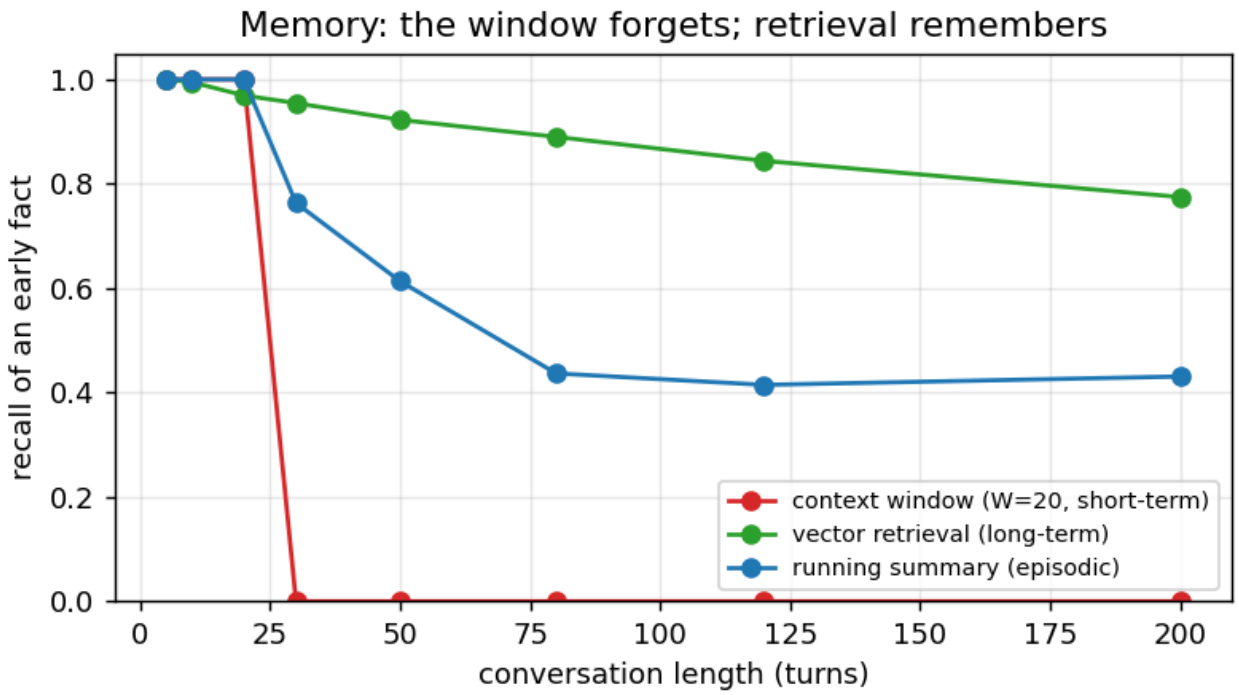


Listing 7 — Memory: window vs retrieval vs summary

Probing recall of an early fact as a conversation lengthens: the context window forgets it off a cliff, vector retrieval holds it, and a running summary settles at a lossy floor.

```
"""Listing 7: agent memory -- short-term vs long-term vs episodic. As a conversation
grows, a fixed
CONTEXT WINDOW (short-term memory) can only hold the last W turns, so a fact introduced
early is
forgotten the moment it scrolls out. LONG-TERM memory stores every turn in an external
vector store
and RETRIEVES the relevant ones on demand, so an early fact stays recoverable
regardless of
conversation length (with slow decay as more distractors accumulate). EPISODIC memory
keeps a
running compressed SUMMARY of old turns -- cheaper than full retrieval but lossy, so
recall is
constant but imperfect. We probe recall of one early fact as the conversation
lengthens."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt, os
FIG="figures/ch25"; os.makedirs(FIG,exist_ok=True); rng=np.random.default_rng(0)
d=48; W=20                                  # context window of 20 turns
def probe(T, mode, t0=3, n=1500):
    hit=0
    for _ in range(n):
        facts=rng.standard_normal((T,d)); facts/
=np.linalg.norm(facts,axis=1,keepdims=True)
        q=facts[t0]+0.35*rng.standard_normal(d); q/=np.linalg.norm(q)   # query about
the early fact
```

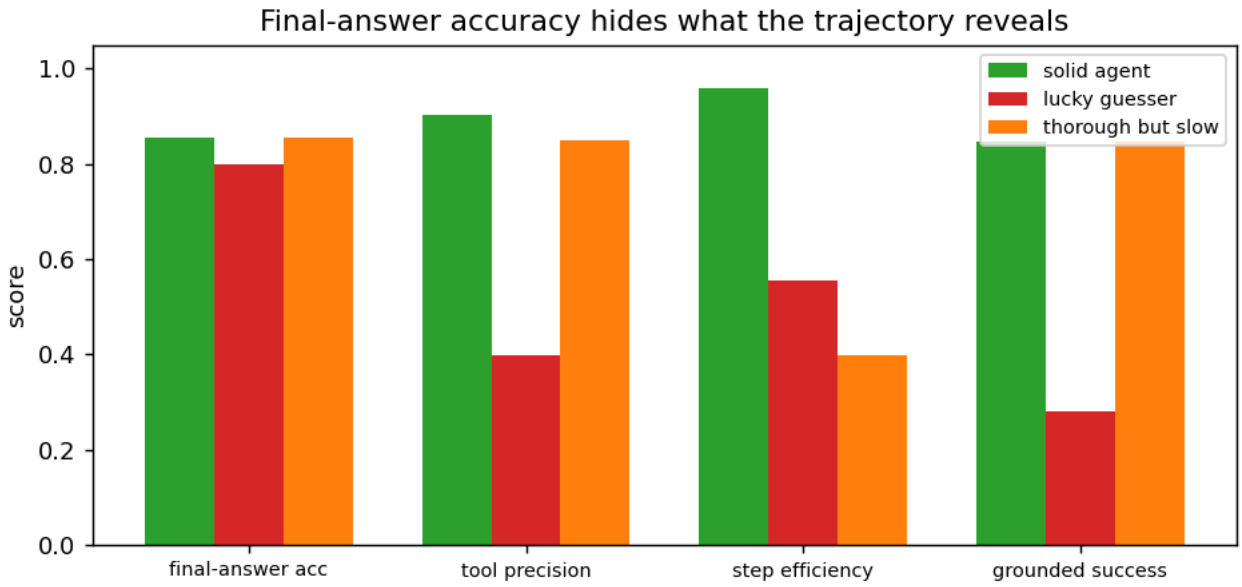
```
        if mode=="window":
            hit += 1 if (T-t0)<=W else 0                # in context iff within last W
turns
        elif mode=="retrieval":
            top=np.argsort(-(facts@q))[:5]              # retrieve top-5 from ALL
history
            hit += 1 if t0 in top else 0
        elif mode=="summary":
            # old turns (< T-W) survive only as a lossy summary: recovered w.p. that
decays with how
            # much has been compressed into one summary slot
            if (T-t0)<=W: hit+=1
            else: hit += 1 if rng.random()< max(0.42, 0.85 - 0.008*(T-W)) else 0
    return hit/n
Ts=[5,10,20,30,50,80,120,200]
win=[probe(T,"window") for T in Ts]; ret=[probe(T,"retrieval") for T in Ts];
summ=[probe(T,"summary") for T in Ts]
for T,a,b,c in zip(Ts,win,ret,summ): print(f"len={T:3d}: window={a:.2f} retrieval={b:.
2f} summary={c:.2f}")
plt.figure(figsize=(6.2,3.6))
plt.plot(Ts,win,"o-",color="#d62728",label=f"context window (W={W}, short-term)")
plt.plot(Ts,ret,"o-",color="#2ca02c",label="vector retrieval (long-term)")
plt.plot(Ts,summ,"o-",color="#1f77b4",label="running summary (episodic)")
plt.xlabel("conversation length (turns)"); plt.ylabel("recall of an early fact");
plt.ylim(0,1.05)
plt.title("Memory: the window forgets; retrieval remembers")
plt.legend(fontsize=8); plt.grid(alpha=.3); plt.tight_layout()
plt.savefig(f"{FIG}/l7_memory.png",dpi=130); print("saved l7_memory.png")
```



Listing 8 — Trajectory evaluation and observability

Three agents with near-identical final-answer accuracy are separated only by trajectory metrics — tool precision, step efficiency, and grounded success — the reason observability exists.

```
"""Listing 8: evaluating agents needs the TRAJECTORY, not just the final answer. A
final-answer
accuracy metric is blind to HOW the answer was reached: a lucky guesser can score well
on final
answers while calling the wrong tools and skipping the reasoning (not reproducible,
unsafe), and a
thorough agent can be correct but wildly inefficient (slow, expensive). Observability/
tracing exists
to capture the whole trajectory so these are measurable. We simulate three agent
profiles and score
four metrics -- final-answer accuracy, tool-call precision, step efficiency (optimal/
actual steps),
and grounded success (correct AND via a valid trajectory) -- to show final-answer
accuracy alone
ranks all three as similar while the trajectory metrics separate them."""
import numpy as np, matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt, os
FIG="figures/ch25"; os.makedirs(FIG,exist_ok=True); rng=np.random.default_rng(0)
def simulate(profile, n=6000):
    fa=tp=eff=grnd=0
    p_final,p_tool,eff_ratio = profile
    for _ in range(n):
        opt=int(rng.integers(2,6))
        correct = rng.random()<p_final
        ncalls=max(1,int(round(opt/max(eff_ratio,0.2))))         # inefficient -> more
calls
        good_calls=np.sum(rng.random(ncalls)<p_tool)
        tool_prec=good_calls/ncalls
        step_eff=min(1.0,opt/ncalls)
        valid_traj = correct and (tool_prec>=0.5)                # grounded = right
answer via mostly-right calls
        fa+=correct; tp+=tool_prec; eff+=step_eff; grnd+=valid_traj
    return np.array([fa,tp,eff,grnd])/n
profiles={"solid agent":(0.85,0.90,0.90),
          "lucky guesser":(0.80,0.40,0.55),
          "thorough but slow":(0.85,0.85,0.40)}
labels=["final-answer acc","tool precision","step efficiency","grounded success"]
res={k:simulate(v) for k,v in profiles.items()}
for k,v in res.items(): print(f"{k:20s}: "+"  ".join(f"{l}={x:.2f}" for l,x in
zip(labels,v)))
x=np.arange(4); w=0.25; col={"solid agent":"#2ca02c","lucky
guesser":"#d62728","thorough but slow":"#ff7f0e"}
fig,ax=plt.subplots(figsize=(7.4,3.6))
for i,(k,v) in enumerate(res.items()):
    ax.bar(x+(i-1)*w,v,w,label=k,color=col[k])
ax.set_xticks(x); ax.set_xticklabels(labels,fontsize=8); ax.set_ylabel("score");
ax.set_ylim(0,1.05)
ax.set_title("Final-answer accuracy hides what the trajectory reveals");
ax.legend(fontsize=8)
plt.tight_layout(); plt.savefig(f"{FIG}/l8_eval.png",dpi=130); print("saved
l8_eval.png")
```



Interview questions and answers

Q1. What is an AI agent, and how does it differ from a plain LLM call or a chatbot?

An agent uses an LLM as the CONTROLLER of a loop rather than a one-shot text generator: it observes a state, decides an action (call a tool, read/write memory, or answer), executes it, observes the result, and repeats until done. A plain LLM call maps input to output once; a chatbot adds conversational memory but still just responds. An agent adds autonomy over MULTIPLE steps and the ability to ACT on the world through tools. The engineering shift is from generation to control: the interesting quantities become success as a function of step budget, per-step reliability, environment predictability, and total trajectory cost.

Q2. Describe the core agent loop and why per-step errors compound.

The loop is perceive-decide-act: given a goal and context, the model emits an action, an external system executes it and returns an observation, the observation is appended to context, and the loop repeats until a terminal answer or a budget is hit. Because it is sequential and stochastic, errors compound: a task needing h correct steps in a row succeeds with probability roughly p^h where p is per-step reliability. Listing 1 shows this — success plateaus at 0.90, 0.81, 0.73 for one, two, and three hops at p=0.9. So long-horizon agents are fragile, and agent engineering is largely about raising p (better tools, verification) or shortening the horizon (decomposition).

Q3. What is ReAct?

ReAct (reason + act) interleaves a Thought, an Action (a tool call), and an Observation on each iteration, deciding the next action from the latest observation. It grounds the model in external results step by step, which converts questions the model cannot answer from its weights into answerable ones and lets it handle problems whose steps are not known in advance. It is maximally adaptive because every step reconsiders the situation, but it costs one LLM call per step and can wander without a global plan. It is the default single-agent architecture and the bridge from prompting to agents.

Q4. Why does a ReAct agent need a loop, and what are its two failure modes (Listing 1)?

Because facts and intermediate results the agent needs live outside its weights and must be fetched step by step; a multi-hop question requires chasing references in sequence, which cannot be done in a single forward pass. Listing 1's success surface shows the two failure modes cleanly: success is exactly zero until the step budget reaches the number of hops (you cannot follow h references in fewer than h steps — a BUDGET failure), and then it plateaus at p^h (compounding per-step error — a RELIABILITY failure). No amount of budget fixes low reliability, and no reliability fixes an insufficient budget.

Q5. Compare plan-and-execute with ReAct. When does each win (Listing 2)?

Plan-and-execute front-loads reasoning into one upfront plan, then executes the steps cheaply — low cost and latency, and a legible plan to inspect — but it is brittle because a pre-written plan assumes the world cooperates, so any mid-run surprise derails it. ReAct re-decides every step from the latest observation — robust to surprises but pays a call per step. Listing 2 sweeps environment surprise: at low surprise, plan-and-execute matches ReAct at a fifth of the cost (1 call vs ~5); as surprise rises, plan collapses (1.00 to 0.03) while ReAct stays robust (0.68). Plan when the environment is predictable and cost matters; react when it is not.

Q6. What is the replanning hybrid and why is it often the production choice?

Plan-and-execute with replanning runs the upfront plan but, when a step's observation contradicts the plan's assumption, pays for one replan and continues — instead of blindly failing (pure plan) or replanning at every single step (ReAct). Listing 2 shows it captures most of ReAct's robustness (success 0.78 at high surprise vs ReAct's 0.68) at a fraction of the cost (rising only to ~3 calls vs ReAct's ~5). It is the pragmatic middle: cheap and legible when the environment behaves, adaptive when it does not. Most production agent frameworks default to some form of plan-then-adapt for this reason.

Q7. What is reflection (Reflexion), and what determines whether it helps (Listing 3)?

Reflection has the agent critique its own previous attempt and retry, ideally learning from what went wrong. Its usefulness is entirely conditional on the quality of the feedback signal. Listing 3 compares reflection with a real VERIFIER (unit test, tool error, ground-truth check) against blind self-critique: with a verifier, cumulative success climbs 0.30 to 1.00 over a few rounds with diminishing returns; with self-critique only, success stays flat at 0.31 and blind reflection sometimes OVERWRITES a correct answer. The lesson: reflection is a way to EXPLOIT an error signal, not to CREATE one — an agent that cannot tell whether it succeeded gains nothing from reflecting.

Q8. How do tool use and function calling work?

The model is shown tool schemas (name, description, typed arguments) and, instead of prose, emits a structured call naming a tool and filling its arguments; an external runtime executes the call and returns the result, which the agent observes and continues from. This is how an agent acts on the world and grounds itself in facts outside its weights — search, code execution, database queries, APIs. Function calling is the productized form of the ReAct action step, and modern models are fine-tuned to emit these structured calls reliably and to know when a call is needed versus answering directly.

Q9. Why does tool selection degrade with many tools, and what fixes it (Listing 4)?

With a handful of tools the model picks reliably, but as the toolset grows, choosing the right one becomes a needle-in-a-haystack over a long, confusable in-context list, and attention over that list is noisy. Listing 4 measures selection accuracy falling from 1.00 at 2 tools to 0.07 at 128. The fix is tool RETRIEVAL: embed the query, retrieve the top-m most relevant tool schemas, and let the model choose only among those — bounding the choice keeps accuracy far higher (0.52 at 128 tools). It is RAG applied to the tool catalog, and the standard pattern once an agent has more than a few dozen tools.

Q10. How do you guarantee a valid function call, and why does it matter end-to-end?

Compile the tool's argument schema into a token-level grammar and mask any token that would violate it — constrained decoding (Chapter 23) — so the call is valid JSON matching the schema by construction, versus unconstrained generation where a single bad token breaks the parse and validity collapses as per-token error rises. Listing 4 shows why both selection and validity matter: an end-to-end successful call needs the right tool AND valid arguments, so the probabilities multiply — naive-plus-unconstrained lands at 0.04 while retrieve-plus-constrained reaches 0.57. Constrained decoding guarantees form, not truth: arguments can still be schema-valid but wrong.

Q11. What is the Model Context Protocol (MCP) and what are its primitives?

MCP is an open protocol that standardizes the interface between an LLM application (the host/client) and external capability servers over a uniform JSON-RPC transport. Its three primitives are TOOLS (callable functions), RESOURCES (readable data the model can pull in as context), and PROMPTS (reusable templates). The point is decoupling: before MCP every framework wrote bespoke glue for every integration; with MCP a server (filesystem, GitHub, a database) is written once and any MCP-compatible client can use it. Think USB-C for tool use — a common connector — which is why it is becoming the default way agents acquire capabilities.

Q12. Chains vs graphs: what do LangChain and LangGraph each model?

A chain is a directed SEQUENCE of steps (prompt, LLM, parse, tool, ...) composed so each output feeds the next — enough for linear pipelines but unable to express loops or conditional branching cleanly. LangGraph generalizes to a GRAPH: nodes are functions that read and write a shared, typed STATE object, and edges (including conditional edges that route based on state) connect them, so cycles (retry until a check passes), branches (route by intent), and fan-out/fan-in (parallel workers) are first-class. Modeling an agent as an explicit state machine makes complex control flow legible and testable, which is why graph frameworks displaced ad-hoc loops for anything nontrivial.

Q13. What is state in LangGraph, and why do conditional edges and cycles matter?

State is a shared, typed object that every node reads from and writes to as the graph runs — it carries the conversation, intermediate results, retrieved documents, loop counters, and flags. Conditional edges route control based on that state (for example, verify -> retrieve again if a check failed, else -> end), which is what lets a graph express a retry LOOP, a branch by intent, or an early exit — control flow a linear chain cannot. Cycles with a loop cap implement iterative refinement (retrieve-generate-check-repeat) safely. Explicit state plus conditional routing is the whole reason to model an agent as a graph rather than a straight-line chain.

Q14. What is checkpointing and why is it essential (Listing 5)?

Checkpointing persists the graph's state after each node so a run can RESUME from the last completed node instead of restarting after an interruption — a crashed tool, a rate limit, a process restart, or a human-in-the-loop pause. Listing 5's from-scratch executor shows why it is not optional: without checkpointing, a crash restarts the whole graph and recomputes every prior node, so total work explodes from 7 node executions with no crashes to 808 at a 50% per-node crash rate; with checkpointing it stays bounded at 20. The same persisted state also enables human-in-the-loop approval and time-travel debugging, which is why durable state is the backbone of production agent frameworks.

Q15. How does checkpointed state enable human-in-the-loop and debugging?

Because the full state is persisted at each node, the graph can PAUSE at a node (say, before a high-stakes tool call like sending an email or spending money), surface it to a human for approval, and RESUME from that exact checkpoint once approved — no recomputation. The same mechanism enables time-travel debugging: replay the run from any saved checkpoint, inspect or edit the state, and branch to explore alternatives. This turns an opaque, non-reproducible loop into an inspectable, resumable process, which is why durable checkpointing is treated as core infrastructure rather than a convenience.

Q16. When does a multi-agent system beat a single agent (Listing 6)?

For two separable reasons. Decomposition under context limits: a single agent handling many sub-tasks in one growing context suffers interference and its per-sub-task reliability degrades — Listing 6's single agent sinks from 0.92 on one sub-task to 0.04 on ten — whereas a supervisor routing each sub-task to a FRESH worker with a clean, focused context keeps success far higher (0.43 at ten). Aggregation: independent agents voting is a Condorcet effect that lifts a 0.62-per-agent decision to 0.79 at eleven agents. Multi-agent is justified when a task genuinely decomposes or benefits from diverse votes — not as a default, because each agent multiplies cost.

Q17. Explain the supervisor-worker pattern.

A supervisor (orchestrator) agent decomposes a task, routes each sub-task to a specialized worker agent with its own clean context and instruction set, and aggregates the workers' results — optionally looping if a sub-task needs rework. The benefit is that each worker operates on an uncluttered context focused on one job, avoiding the interference that degrades a single long-context agent (Listing 6: 0.43 vs 0.04 on a ten-sub-task job). The costs are orchestration overhead, more model calls, and a coordination protocol. Variants include hierarchical teams (supervisors of supervisors) and role-specialized workers (planner, coder, critic, researcher).

Q18. How does agent debate/voting work and what is its precondition?

Multiple agents independently answer the same question and a majority vote (or a round of mutual critique, in debate) selects the result — a Condorcet-jury effect, the agent-level version of self-consistency. Listing 6's vote over k agents, each correct 62% of the time, rises to 0.79 at eleven agents. The precondition is exactly Condorcet's: each agent must be better than chance AND their errors must be INDEPENDENT. Correlated agents — the same model with the same prompt — do not decorrelate and gain little; diversity (different models, prompts, or tools) is what makes voting pay off. It trades compute linearly for reliability.

Q19. What are the risks and costs of multi-agent systems?

Every extra agent multiplies token cost and latency, so the compute bill can dwarf a single well-prompted agent. Communication between agents adds surface where errors and hallucinations PROPAGATE — one agent's confident mistake becomes another's input — and coordination requires a protocol that can deadlock, loop, or lose information in hand-offs. Debugging is harder because failures are distributed across a conversation among agents. So multi-agent is warranted when a task genuinely decomposes or benefits from diverse aggregation, and a single agent with good tools and memory is the right default otherwise — complexity should be added only when a measured need justifies it.

Q20. Describe the types of agent memory (Listing 7).

Short-term / working memory is the context window plus a scratchpad within a task: fast and fully attended but bounded and volatile — Listing 7 shows a fixed window's recall of an early fact falling off a cliff to zero once the conversation exceeds the window. Long-term memory writes everything to an external store and retrieves relevant pieces on demand (RAG over the agent's history), holding recall at 0.77 even at 200 turns. Within long-term, SEMANTIC memory holds facts, EPISODIC memory holds records of past interactions/outcomes to replay, and PROCEDURAL memory holds learned skills. A running summary is a cheap lossy episodic form that settles at a recall floor (~0.42).

Q21. Why is the context window called short-term memory, and what breaks?

Because it holds only the tokens currently in front of the model — fully attended and fast, but finite and volatile: anything beyond the window is simply gone, with no gradual decay, just a hard cutoff. Listing 7 probes recall of a fact from early in a conversation and finds a cliff — perfect recall until the conversation length exceeds the window, then zero, because the fact scrolled out. Even within the window, the lost-in-the-middle effect (Chapter 22) under-weights the middle. So the window alone cannot support long interactions or durable knowledge, which is why external long-term memory exists.

Q22. How does long-term (retrieval) memory work, and what is its failure mode?

Every turn or fact is written to an external vector store; when the agent needs past information it embeds the current query and retrieves the most relevant stored items, injecting them into the context — RAG (Chapter 24) applied to the agent's own history. Listing 7 shows it holds recall at 0.77 even at 200 turns, versus zero for a window. Its failure modes are the retrieval ones: recall decays slowly as accumulated history adds distractors, irrelevant retrieved memories can DISTRACT the model, and a write/forget policy is needed to avoid unbounded growth and stale or contradictory memories. Managing what to store, summarize, retrieve, and forget is the hard part.

Q23. Distinguish semantic, episodic, and procedural long-term memory.

Semantic memory is facts and knowledge — the agent's stored documents and learned information, usually the vector-retrieval store. Episodic memory is records of specific past interactions and their outcomes — what was tried, what happened — which the agent can replay to inform new tasks, the basis of experience-based improvement (and what Reflexion writes its reflections into). Procedural memory is learned skills and routines — how to do recurring things — often baked into the system prompt, tools, or fine-tuned behavior rather than retrieved. Production agents combine all three with the short-term window: the window for the active task, summaries to compress the recent past, and the stores for durable recall.

Q24. Why is evaluating an agent harder than evaluating a single LLM call (Listing 8)?

Because success is a property of the whole TRAJECTORY, not a single output — the same final answer can come from a sound process or a lucky accident, and final-answer accuracy alone cannot tell them apart. Listing 8 scores three agents with near-identical final-answer accuracy (0.85/0.80/0.85) on four metrics; the trajectory metrics separate them: the lucky guesser has tool precision 0.40 and grounded-success 0.28 (unreproducible, unsafe wins), and the thorough agent has step-efficiency 0.40 (2.5x the necessary calls). Neither pathology shows in the final-answer number. You must evaluate HOW the answer was reached, not just whether it was right.

Q25. What agent metrics matter, and what is observability's role?

Componentize along the trajectory: task success (end-to-end), step/trajectory efficiency (steps vs optimal, token and dollar cost), tool-call accuracy (right tool, valid arguments, correct results), grounding/faithfulness of intermediate reasoning, and RAGAS-style metrics (Chapter 24) for any retrieval. Observability — capturing the full trace of every thought, tool call, argument, observation, and cost — is the foundation that makes these measurable, which is why tracing tools (LangSmith, LangFuse, OpenTelemetry-based tracing) are standard infrastructure. Evaluation combines curated task suites, programmatic checks where possible (unit tests, exact match, tool-error signals), and LLM-as-judge for open-ended steps, with its known biases.

Q26. What are the main agent failure modes and how do you engineer around them?

Compounding per-step error over long horizons (raise per-step reliability with better tools and verification, or shorten the horizon via decomposition); getting stuck in loops or running out of budget (step caps, loop detection, budget-aware stopping); brittle plans (replanning); wrong tool or invalid arguments (tool retrieval and constrained decoding); forgetting (external memory); hallucinated observations and error propagation (grounding, verification, human-in-the-loop for high-stakes actions); and prompt injection through retrieved/tool content (Chapter 23 defenses). Reliability engineering is mostly about adding trustworthy signals (verifiers, schemas, checkpoints) and bounding the blast radius of any single wrong step.

Q27. When should you NOT build an agent?

When a simpler architecture solves the problem, which is most of the time. If a task is a fixed pipeline with no branching, a chain or a single well-prompted call is cheaper, faster, and more reliable than an autonomous loop. Agents add latency, cost (many calls), and failure surface (compounding errors, loops, injection), so they are justified only when the task genuinely needs autonomy over multiple steps, tool use whose sequence is not known in advance, or adaptation to unpredictable environments. The engineering discipline is to reach for the least autonomy that works — deterministic code, then a chain, then a constrained agent, then a full multi-agent system — and add complexity only when a measured need justifies it.

Q28. How do agents relate to RAG and prompting, and what is agentic RAG?

Agents build directly on the prior two chapters. Prompting (Chapter 23) supplies the mechanisms an agent's controller runs on — chain-of-thought for its reasoning steps, ReAct for its act-observe loop, constrained decoding for its tool calls, and self-consistency for its voting. RAG (Chapter 24) is the agent's semantic long-term memory and one of its most important tools: retrieval grounds the agent in external facts. AGENTIC RAG closes the loop between them — instead of a single fixed retrieve-then-read pass, an agent DECIDES whether to retrieve, reformulates the query, retrieves iteratively, evaluates the results (corrective RAG), and chains multi-hop lookups, using the ReAct loop to drive retrieval. It is where conditional, evaluated, tool-driven retrieval (the direction advanced RAG was already heading) becomes a full agent behavior.